

Fading Strategies for Parsons Problems in Intermediate Classrooms

Serena Caraco
caraco@berkeley.edu
University of California, Berkeley
Berkeley, CA, USA

Nelson Lojo
nelson.lojo@berkeley.edu
University of California, Berkeley
Berkeley, CA, USA

Armando Fox
fox@berkeley.edu
University of California, Berkeley
Berkeley, CA, USA

Abstract

Faded Parsons Problems (FPPs) are a variant of Parsons Problems in which students reconstruct short pieces of code by filling in blanks (“fades”) and rearranging pre-written scrambled lines of code. We are particularly interested in FPPs’ demonstrated superiority over code-writing or code-comprehension exercises for scaffolding the teaching of advanced programming concepts. However, little is known about how the choice of what to fade (i.e. which tokens to blank out in the scrambled code lines) affects an FPP’s difficulty and efficacy, and existing Parsons Problems research does not address advanced classrooms.

To inform the choice of what to fade, and potentially allow automated fading of a reference solution for formative assessments, we devise a conceptual guide for constructing FPP “fading strategies” informed by guidelines from prior work as well as our own experience. Using this guide, we generate three strategies that are variably aware of the context of the program text they fade. We compare the strategies using a between-subjects study in which students solve FPPs around an advanced software engineering topic. We find that the difficulty and efficacy of the problems, as measured by time on task, qualitative surveys, and long-term retention, does not appear to depend on context-awareness, but also find that random choices are, in some senses, “more obfuscating” than our other strategies. We conclude that our simplest strategy suffices for the future study of automatic fading of FPPs scaffolding advanced topics.

CCS Concepts

- **Applied computing** → **Interactive learning environments**; • **Social and professional topics** → *Software engineering education*;
- **Theory of computation** → Algorithm design techniques.

Keywords

Faded Parsons Problems, Fading Strategies, Software Engineering Education, Ruby RSpec, Autograding, Language-Agnostic

ACM Reference Format:

Serena Caraco, Nelson Lojo, and Armando Fox. 2025. Fading Strategies for Parsons Problems in Intermediate Classrooms. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2025)*, June 27–July 2, 2025, Nijmegen, Netherlands. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3724363.3729062>



This work is licensed under a Creative Commons Attribution 4.0 International License. ITiCSE 2025, Nijmegen, Netherlands

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1567-9/2025/06
<https://doi.org/10.1145/3724363.3729062>

1 Introduction, Motivation, Contributions

Parsons Problems (PPs) test a student’s ability to reconstruct short scripts from a number of pre-written, scrambled lines of code, or *codelines*. One variant, *Faded Parsons Problems* (FPPs) [13], embeds fill-in-the-blank-style *fades* in the standard codelines, so that students must both complete the codelines and arrange them correctly. FPPs are particularly effective in scaffolding more advanced programming patterns [11], and because they are amenable to automated grading with sophisticated feedback even with only partial solutions [2, 4, 15], they are attractive for teaching at scale.

The difficulty of solving FPPs is certainly affected by the amount of fading: in the limit, if every token on a codeline is faded, the FPP effectively becomes an open-ended code-writing exercise. In this paper we investigate the extent to which FPP difficulty and efficacy is also affected by the choice of *which* tokens to fade: variable names, control-flow keywords, function arguments, and so on.

The scenarios of interest to us share two properties. First, we assume the students are already familiar with the basic syntax of the language used in the exercises, so teaching language syntax is a *non-goal*. This differs from many uses of PPs in introductory courses, in which scaffolding the teaching of syntax is part of the goal. Second, and related, we assume that the focus of the exercise is to learn how to express or implement an advanced programming concept or pattern requiring the use of external libraries, such as authoring unit tests (as in the case study described in this paper).

This distinction is important because it speaks to the *nature of the cognitive load* that we want the exercises to include and exclude. For example, as proposed in [11], in which the programming pattern in question involves the structure of unit tests, the goal of such exercises is to get the mechanics of the library function call “out of the way”—types and order of arguments, manipulating the returned object, and so on—while students focus on the conceptual challenges associated with implementing the programming pattern; and then slowly increase the fading so students gradually start to grapple with these mechanics as well. In other words, in these exercises, the primary cognitive load on the students should stem from grasping the *semantics* of instantiating a programming pattern in the service of some advanced programming concept, rather than the *syntax* of the language itself.

To investigate the effect on difficulty of different “fading strategies” in such exercises, this paper offers the following contributions:

- (1) We outline a *conceptual guide* for determining what to fade, based on both syntactic and semantic considerations of the code. Specifically, in view of our assumption that teaching language syntax is a non-goal, we wish to avoid fading that introduces undesirable cognitive load in which students are distracted by the effect of fading on language syntax, rather than focusing on the semantics of the problem in the exercise.
- (2) We develop three context-aware fading strategies: given as input a reference solution and an approximate fraction of *how*

much to fade, the strategies make choices about *what* to fade, and instantiates an FPP ready for assigning to students.

- (3) We conduct an IRB-approved between-subjects user study ($n = 95$) in which software engineering students complete FPPs related to unit-test creation, with different subgroups of students being exposed to different fading strategies. We examine both students' perceived effectiveness and difficulty of the FPPs (via NASA TLX), and their material retention measured two weeks after completing the exercises.

After reviewing related work, we describe our guide for building fading strategies and the strategies we made with it. We then detail our study design, administration, and results. While we find that our fading strategies are generally equivalent, we also demonstrate that there are subtle disadvantages to choosing more random strategies. Finally, we conclude with a recommendation to choose a strategy that adheres to our proposed guidelines and prefers context-awareness over a purely random choice, while leaving the strategy's specifics to the instructor's discretion.

2 Related Work

The effectiveness of PPs in scaffolding programming concepts is well-studied [5, 12, 16], and PP *variants*, which often add additional interactivity to the PP format, have demonstrated efficacy in a variety of contexts [3, 4, 10]. For example, variants called Adaptive Parsons Problems and Micro Parsons make use of real-time personalized feedback, to great effect [4, 15]. However, where PPs and these variants may be trivially derived from the solutions to open-ended coding problems, FPPs require the manual introduction of fading. At UCB, we use a learning-management system capable of generating practice problems on-demand. With automatic fading, FPPs could be generated on-demand, just like the other PP variants.

To make automatic fading sensible, we must first understand the impact fading has on the difficulty of an FPP. The single known work exploring this question, Fromont et al. [7], performed an initial exploration of the relative difficulty engendered by different fadings of FPPs, in the context of introductory-level programming in C. The authors propose several potential *fading strategies* that define a system for selecting what to fade. They evaluate three of these strategies alongside the standard PPs as a control ($n=915$). By analyzing subjective surveys, solving time, and number of attempts, they order the strategies from most to least difficult: (1) the "Conditional" strategy fades complete expressions that return boolean values, (2) the "Variable" strategy fades variable names, (3) the "Operator" strategy fades binary operators (e.g. &&, », /=), and finally (4) the unfaded control.

While the basic strategies do make use of some AST structure and type information, they make no attempt at intelligently selecting one fading target over another, choosing while blind to the semantic and syntactic context of the fade. For example, it is not uncommon for functions and keywords of interest to appear repeatedly throughout a problem's prompt, or for important variables to appear repeatedly throughout a problem's codelines; conversely, fading a critical keyword that only appears once in the entire problem text could render a problem unsolvable. These are simple indicators of interesting semantics that could be put to use in automatic selection from among many candidates of fades.

Figure 1: Prompt and Solution to RSpec Exercise Problem #1

A student must write unit-tests for the GiftCard class.

```
describe GiftCard do
  it 'fails to instantiate with negative balance' do
    expect { GiftCard.new(-1) }.to raise(ArgumentError)
  end
  it 'successfully assigns a positive balance on init' do
    @gift_card = GiftCard.new(20)
    expect(@gift_card.balance).to eq(20)
  end
end
```

Furthermore, these basic strategies struggle to operate in programming languages and paradigms beyond imperative C. For example, UCB's software engineering course uses FPPs to scaffold unit test creation in Ruby, but applying the Fromont strategies often resulted in far too many targets or too few. Figure 1 lists the unfaded code-lines for the first question on a worksheet for this course. Applying Fromont's strategies to it yields 0 Operator fades, 0 Conditional fades, and depending on the implementation, 1-12 Variable fades. Across the entire worksheet (54 codelines across 5 problems), the strategies yield 0, 4, and 22-90 fades, demonstrating the unreliability of the basic strategies for automatic use in this context.

In the following section, we extend Fromont to create a theoretical foundation for automatic fading; we provide a conceptual guide for the creation of automatic fading strategies and then use this guide to create syntax- and semantics-aware strategies.

3 A Conceptual Guide for Strategy Creation

We formalize a *fading strategy* as a procedure to generate *fading candidates* (potential text to replace with blanks within an FPP code-line), which concludes with *candidate selection* (the procedure for selecting fades from candidates). We begin with goals and guidelines for creating fading strategies, and then we give examples of three new strategies.

3.1 Goals & Guidelines for New Fading Strategies

In this section, we detail the guidelines summarized in Table 1. The intent of our guidelines is to align with the spirit of Parsons Problems generally — reducing the cognitive overhead of irrelevant syntax and semantics. Following the approach of previous work [7], we phrase the guidelines in terms of tokens, symbols, and expressions. Setting the fade granularity in these terms avoids simple problems that could arise from naïve selection, such as splitting a language-specific keyword into pieces or unintentionally fading meaningful whitespace. The guidelines are set up to achieve three goals:

- Legible: make fades easier to identify and reason about
- Reliable: prevent autograder context from leaking
- Predictable: elicit predictable solutions and error messages

Note that these goals function independently of the particulars of an FPP's UI, focusing only on the text of the fades themselves. The following lists how each guideline contributes to the goals:

- #1: "A faded control keyword must be the only fade on a line." (Legible.) Because lines with control keywords often effect a

Table 1: Guidelines and Examples for New Universal Fading Strategies

Guideline	Unfaded Codeline	Valid Fade	Invalid Fade
1. A faded control keyword must be the only fade on a line	if foo(arg1) do	_ foo(arg1) do	_ foo(_) do
2. Fades should be separated by more than whitespace	x = 3	_ = _	_ _ 3
3. Never fade scoping syntax symbols, e.g. } : ; (\ global do	if foo(arg1) do	if _(arg1) do	if _ arg1) _
4. Do not fade the first appearance of a name in the problem	x m * x + b	x m * _ + b	_ m * x + b
5. A faded symbolic operator or delimiter must include either all or none of its operands in its fade. A faded unary prefix operator should always include its operand.	bar(10, 20, 30)	bar(_)	bar(_, 30)
	Pair::new	Pair::_	_ new
	n + 1	n _ 1 or simply _	_ 1
	&42	_	_ 42

program’s control-flow graph, we assume they come with an increased cognitive overhead when solving. Thus, if a control keyword is faded, the surrounding context should remain so the proper control structure may be inferred.

- #2: “Fades should be separated by more than whitespace.” (Legible, Predictable.) Fades separated by whitespace alone are often redundant because whitespace is allowed in blanks, decreasing legibility and growing the space of correct answers.
- #3: “Never fade scoping syntax symbols.” (Legible, Reliable, Predictable.) Missing scoping symbols can cause parsing to fail before an AST is constructed. This may produce unintelligible error messages or, in the worst case, leak information about the autograding context. Furthermore, in many IDEs, these symbols are automatically inserted and highlighted, which can make their absence difficult to detect.
- #4: “Do not fade the first appearance of a name in the problem.” (Legible, Reliable.) In most languages, the first appearance of a name is part of its definition. In the extreme case, fading every instance of a name makes the fades trivial because any consistent name would suffice. In more common cases, allowing definitions to have arbitrary names can foil autograders, including some of those used at UCB.
- #5: “A faded symbolic operator or delimiter must include either all or none of its operands in its fade. A faded unary prefix operator should always include its operand.” (Legible, Reliable, Predictable.) Such invalid fades left blank often create subtle bugs or may cause parsing to fail altogether, in both cases reproducing the issues in guideline three.

While other goals and guidelines may certainly exist, we claim these should not be modified for three reasons. First, all seven of Fromont’s basic strategies (including the three they did not evaluate) retroactively adhere to these guidelines, despite being empirically derived from a dataset of many different FPP examples in C, demonstrating the guidelines’ linguistic flexibility and respect for existing norms. Second, taken in conjunction, the guidelines provide guarantees about the outcomes of arbitrary candidate selection processes, rendering automatic fading more reliable in production (the next section elaborates this point). Finally, the guidelines often make incomplete problems appear syntactically incorrect *but still parsable*, meaning that students would see the same error messages they would usually see when dealing with actual code in an IDE.

3.2 Three Novel Context-Aware Fading Strategies

To exemplify using the guidelines and address the limitations of Fromont et al.’s basic strategies, we propose three new strategies on a spectrum of context-awareness — one purely syntax-aware, one semantics-aware, and one neutral hybrid — all visualized in Figure 2.

Mismatched Delimiters, the syntax-aware strategy, attempts to give the student context by way of distinct, syntactic visual cues. As the name suggests, it operates by targeting tokens that are bounded by unequal or mismatched delimiters¹. As an example, consider the snippet “GiftCard.new(20)” — the token new is surrounded by the mismatched delimiters “.” and “(” so it is a candidate for fading, while 20 is surrounded by the *matching* delimiters “(” and “)”, so it is *not*. Ideally, a student seeing a fade there would deduce from the delimiters that a property, or more specifically, a *method* of GiftCard is required, and use other information from the prompt to infer new as the proper choice. In order to maintain a sense of context for the fade, Mismatched Delimiters will never fade the first or last token of a line; because FPP codelines are scrambled, there will be no context provided for what comes before or after a linebreak.

Recall that after generation, fading candidates undergo selection. For Mismatched Delimiters, candidates are laid out on a rectangular grid corresponding to their line and column position. Fades are selected one-at-a-time using a random, spatially distributed set of points, weighting picks by sum of inverse-square distances to all other points.² To avoid biasing the corners of the grid, the distribution uses toroidal distance to allow this distance to “wrap around the edges.” To abide by guideline one, some language-specific control keywords must be marked, but once a set of keywords is set, it is usable for every FPP written in that language.

Note that this strategy’s implementation does not require AST data; in fact, candidate generation requires only a single pass of the codelines, making it an intriguing option for automatic fading.

Prompt-Aware Control Expressions (or *PACE*) is a semantics-aware strategy inspired by Fromont’s “Conditional” strategy. *Control expressions* are those whose value determines control flow, for example:

¹To include languages that use whitespace for function applications (e.g. Ruby, ML), whitespaces *are* considered delimiters, increasing the space of candidates. Note that in such languages, guideline five applies to fading whitespace function application just as it would for parenthetical application, providing strong constraints on fading.

²Fade spatial distribution is useful because degenerate samples (e.g. placing all fades in one codeline) can effectively regress an FPP into a PP. However, even such regressions are legible because of guidelines two & three.

- Boolean-typed control-flow conditions (e.g. the conditions of if-statements, sentinel expressions of loops)
- arguments to case, switch, match, etc.
- conditional early return expressions (e.g. `task.result?`)
- run-time assertion expressions (e.g. `expect(expr).to pred`)
- expressions with monadic operations (e.g. `catch`, `bind`)

Note that operations like `raise` and `return` *always* redirect control flow regardless of their arguments' values, so they are *not* control expressions.

Figure 2: Visualizing Fading Candidates by Strategy for RSpec Exercise Problem #1

(a) Mismatched Delimiters

syntax-aware: targets tokens with unequal or mismatched delimiters, e.g. “.” and “(”

```
describe GiftCard do
  it 'fails to instantiate with negative balance' do
    expect { GiftCard.new(-1) }.to raise(ArgumentError)
  end
  it 'successfully assigns a positive balance on init' do
    @gift_card = GiftCard.new(20)
    expect(@gift_card.balance).to eq(20)
  end
end
```

(b) PACE

semantics-aware: targets control expressions, and weights all sub-expressions by their frequency in the problem prompt (Empty outlines are the least weighted; Green fill is the heaviest. GiftCard, balance appear in the prompt, omitted for brevity.)

```
describe GiftCard do
  it 'fails to instantiate with negative balance' do
    expect { GiftCard.new(-1) }.to raise(ArgumentError)
  end
  it 'successfully assigns a positive balance on init' do
    @gift_card = GiftCard.new(20)
    expect(@gift_card.balance).to eq(20)
  end
end
```

(c) Throwing Darts

neutral hybrid: pools candidates from the other strategies, de-duplicates, and removes all weights

```
describe GiftCard do
  it 'fails to instantiate with negative balance' do
    expect { GiftCard.new(-1) }.to raise(ArgumentError)
  end
  it 'successfully assigns a positive balance on init' do
    @gift_card = GiftCard.new(20)
    expect(@gift_card.balance).to eq(20)
  end
end
```

This strategy also recursively promotes *all valid sub-expressions* of a control expression into candidates themselves. It further includes the definitions of local variables referenced in a control expression, and those definitions' sub-expressions.

As an example of PACE, consider the snippet:

```
u = app.get_user(uid); expect(u.name).to be_truthy
```

Here, `expect` is a runtime assertion function, so it is a candidate, as well as all its argument sub-expressions by promotion: `u`, `name`, and `u.name`. Because `u` is a local variable referenced in a candidate, the definition and its sub-expressions are also promoted: `app`, `get_user`, `app.get_user`, `uid`, and `app.get_user(uid)`. Finally, `be_truthy` sets the conditions for assertion and is therefore a candidate.

To sensibly choose from so many candidates, we construct a semantic means of weighting a candidate's likelihood of selection based on *relevance to the problem*. Generally, a problem's prompt is a source of dense, manually-curated information that is required for a student to succeed. Leveraging this information, the selection algorithm prefers tokens that co-appear most frequently in the problem prompt and codelines, assigning a high weight to candidates that are of focus in a specific problem (and thus mentioned frequently in the problem's prompt or contextual code) and a low weight to candidates that are not. To perform selection, a simple weighted draw suffices, preferring the benefits of semantics over spatial distribution. An example weighted candidate pool for the *Prompt-Aware Control Expressions* (or *PACE*) strategy is depicted in Figure 2b.

The final strategy, *Throwing Darts*, is a hybrid approach. It simply combines the candidates from both strategies, removes weights and duplicates, and enters spatially distributed selection, as in *Mismatched Delimiters*. As a result, the spatial coverage of candidates is generally large, leading to a random, spatially-distributed selection and lending the strategy its name.

4 Study Design

After approval by our institutional Committee for the Protection of Human Subjects, we performed a between-subjects study in a third-year undergraduate software engineering course in the Fall 2024 semester at UCB. Preceding the study, students had received lectures on software testing in RSpec (<https://rspec.info>), a popular unit-testing library derived from JUnit. Out of roughly 150 students in the course, 95 consented to the use of their data for research purposes. To ensure fairness, all students completed all study materials, and none of the study materials were used in calculation of their grades. To prevent instructor bias, instructors were not made aware of the participants' identity until after grading closed, at which time they performed pseudonymization of the data set and delivered it to research staff.

Students were evenly assigned one of the three fading strategies and administered an exercise consisting of 5 FPPs and 4 interleaved expository questions. Similar to past offerings of the course, the exercise was an online, take-home activity through PrairieLearn [14], an online learning management system. Later questions in the exercise revealed answers to their predecessors, and thus each question required 100% correctness before advancing. Students received auto-grader feedback after every attempt, and had unlimited attempts to get full credit. For each FPP, we collected the timestamps and score of each attempt along with the timestamps of every click-and-drag and text-input event. From this data, we can count *FPP interactions*, and

arrange the interactions on a timeline with the problem attempts. Immediately following the exercise, students were surveyed for their subjective experience of the FPP interface using NASA Task Load Index (or *TLX*) [8].

Two weeks after all students had completed the exercise, we delivered a take-home summative questionnaire through Prairielearn. As a summative activity, the questionnaire delivered neither real-time scores nor feedback during solving. Although the questionnaire was designed to measure student ability, as with all study materials, the questionnaire scores were not used in the calculation of course grades. We collected timestamps for every submission for each question to reconstruct a solving timeline.

The questionnaire consisted of 15 multiple choice questions, all designed to yield strong information about a student’s level of understanding in terms of partial credit. To achieve this, we scored 12 questions with the “Singular Choice for Multiple Choice Axiomatic Method” [6] and 3 using the “Multiple True/False Method” [1]. Both of these scoring methods present a student with a question in multiple choice format, and instruct the student to narrow down which choices they think *may be correct*, even if they mark more choices than can possibly be correct. (E.g. a student narrows down to 2 out of 4 choices, gets stuck, and marks them, even though the question specifies only one choice is correct.) In this way, the student gives the graders data about *a student’s certainty* in their knowledge, in addition to that knowledge’s factual correctness.

5 Results

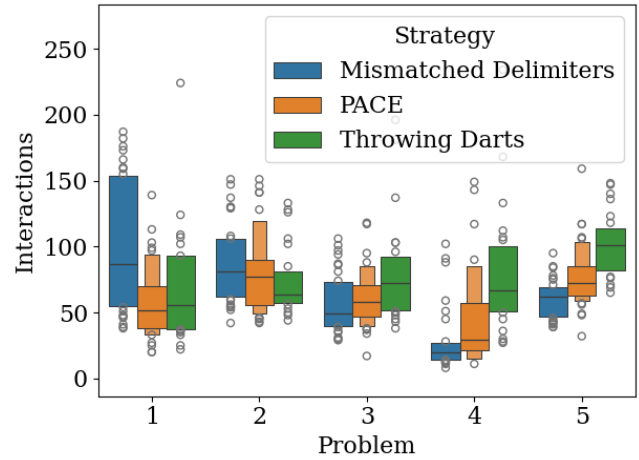
As in Fromont et al., our main statistical tests are Levene’s and Wilcoxon Rank-Sum testing. All data compared in this section failed Levene’s test for heterogeneity of variances, so One-way ANOVA tests are used to determine if *any group* is distinct in a sample. A cross-table of Two-Sided Wilcoxon Rank-Sum tests (*Rank-Sum*) generates a partial ordering of distinct groups. To demonstrate equivalence between groups, we generate a cross-table of Two One-Sided Wilcoxon Rank-Sum tests (*TOSWR*). Because cross-tables can grow large, all reported Rank-Sum and TOSWR p-values are adjusted with the Benjamini-Hochberg procedure [9]. For all tests, the threshold for significance is set at $p = 0.05$ after correction.

5.1 FPP exercise

During the exercise, each time the student submits a solution to the autograder, we call it one *attempt*. Recall that students have unlimited attempts but must get full credit on each problem before they can proceed to the next. We calculate the following three metrics from the collected log data of students solving the FPPs: (1) total solving time for each problem (across all attempts), (2) number of attempts before getting the problem fully correct, (3) interaction frequency, or the number of times a student rearranged a line of code or edited a fade during a given attempt. Note that for each metric, lower values are better.

There is strong evidence that these strategies are equivalent in terms of objective difficulty to complete an FPP. Table 2 shows that all strategies are equivalent with respect to total solving time aggregated over all five FPPs (TOSWR, $p \ll 0.001$). Comparing problem solving time per problem also showed total equivalence on three FPPs (TOSWR, $p \ll 0.001$), and no statistically significant difference on the final two (Rank-Sum, $p > 0.2$). Similarly, total attempts by

Figure 3: Exercise FPP UI Interactions, by Problem



strategy are equivalent (TOSWR, $p \ll 0.001$). As these are the same metrics used in Fromont et al to establish differences in difficulty, we take these findings as strong indicators that objective difficulty is effectively equivalent across all modes of context-awareness.

However, unlike attempts and solving time, FPP interaction counts were distinct across the three strategies (ANOVA, $p \ll 0.001$). In some cases, distinctions remained significant when broken down by problem, visualized in Figure 3. A statistically-significant partial ordering over strategies for each problem is listed in Table 3, indicating a weak hierarchy of interaction frequency. With the exception of Problem 1, PACE was the strongest performer, and tied for the fewest FPP interactions, at worst. Likewise, excluding Problem 1, Throwing Darts was the worst performer, and tied for the most interactions, at best.

Immediately after the exercise, we administered the TLX [8] to estimate the usability of the FPP interface. Using a simple t-test at ($p = 0.05$), all strategies were found to be low physical/temporal demand (TLX, 43.4/46.1) and high performance (TLX, 76.9). The three strategies were neither statistically distinct nor equivalent across all metrics (Rank-Sum, $p > 0.23$; TOSWR, $p > 0.68$, respectively).

5.2 Follow-up Questionnaire

Two weeks after the interactive exercise, 59 of the 95 consented students (62%) completed the follow-up questionnaire. On the questionnaire, the student has one chance to answer each question (vs. unlimited attempts in the exercise) and is not shown their score (vs. getting detailed feedback during the exercise). Questionnaire score is therefore a valid metric of student knowledge.

Data on efficacy indicates general equivalence between strategies, but on individual questions, results are inconclusive. All strategies are equivalent in terms of overall score ($p < 0.01$), but when compared by question, they fail both equivalence (TOSWR, $p > 0.20$) and distinction tests (Rank-Sum, $p > 0.95$). Similarly, Mismatched Delimiters and Throwing Darts are equivalent in total solving time (TOSWR, $p = 0.02$), but once again, all strategies fail distinction tests (Rank-Sum, $p > 0.11$) when grouped by question. We interpret this as rough equivalence for efficacy in terms of retention.

Table 2: A Summary of Findings*Each value is listed with a 95% confidence interval.*

Strategy	Exercise Total Time (minutes)	Questionnaire Total Time (minutes)	Questionnaire Score (%)
Mismatched Delimiters	68.99 [56.78, 81.21]	16.42 [08.59, 24.26]	61.89 [50.52, 73.25]
PACE	71.59 [59.52, 83.65]	22.19 [14.75, 29.63]	59.48 [52.21, 66.75]
Throwing Darts	68.38 [52.33, 84.42]	19.22 [06.60, 31.84]	55.75 [43.16, 68.33]

Table 3: A Partial Ordering of Relative Interaction Frequency

Throwing Darts	Tie		High	High	High
Mismatched Delimiters		Tie	Tie	Mid	Mid
PACE	High			Low	Low
Exercise Problem	#1	#2	#3	#4	#5

Rankings are generated by statistically significant partial ordering, with insignificant differences listed as ties (Rank-sum - $p < 0.05$).

6 Discussion

In contrast with the indicators of objective difficulty and efficacy, interaction frequency data is highly diverse across strategies; we claim this indicates a distinct quality from difficulty or efficacy. The subjective difficulty and efficacy data supports this claim, labeling all strategies as low demand and high performance, further indicating that high interaction frequency is a sign of something unique – what we call *obfuscation*. An obfuscated FPP is not more difficult to solve or less productive of learning gains, instead it requires students to make more UI interactions (and faster) to achieve the same result.

As with difficulty and efficacy, obfuscation may be viewed in aggregate or by problem. Since students took approximately the same amount of time to complete the worksheet across all fading strategies, average time *between interactions* should be inversely proportional to the total number of interactions. Hence, with the shortest average time between interactions, Throwing Darts obfuscated the solution the most of any strategy in aggregate. This generally holds for individual problems as well; looking at Table 3, with the exception of Problem 1, Throwing Darts performed at least as poorly as any other strategy. Conversely, PACE tied for fewest interactions on all problems except #1, and Mismatched Delimiters tied for fewest or second to fewest on all problems. This consistent performance suggests problem-aware fading has an impact that the random, neutral strategy lacks, resulting in higher obfuscation. In light of this, we caution against highly random strategies for candidate generation.

In short, given the general equivalence of our strategies for both in-the-moment difficulty and long-term efficacy, we advise that fading strategies both abide by the guidelines and avoid obfuscation by not heavily relying on randomness for generating fading candidates. From there, the details of the strategy are up to the instructor to suit to the task. For example, we are preparing to complete the existing suite of PrairieLearn FPP autograding features [2, 14] by leveraging the simplicity of Mismatched Delimiters.

6.1 Limitations

This analysis is limited by three major factors. Firstly, the starting population at $n = 150$ sustained attrition between study consent $n = 95$ and completion $n = 59$, paling in comparison to Fromont’s $n = 915$. Combined with the fact that study materials had no bearing on student grades, we attribute the attrition to the timing of the relevant RSpec module within the semester’s exam schedule.

The second limitation is the possibility that strategy efficacy is only equivalent in aggregate, e.g. that some strategies are better for the first problems in a problem set. It is also possible that the amount of fading should vary with time, similar to the scaffolding provided in Adaptive PPs [4]. We encourage further work in this area to see how varying fading with time may affect outcomes.

A final limitation is the unknown cause of the trend-breaking data for Problem 1. This may have occurred because Problem 1 was the first exposure to FPPs for a subset of participants, which would also explain the three-way tie on Problem 2. Alternatively, we know Problem 1 was at least as difficult as any other problem in terms of both attempts and solving time, regardless of FPP familiarity. This difficulty likely stems from the fact that Problem 1 first introduces the GiftCard example, a common class throughout the worksheet.

7 Conclusion

Faded Parsons Problems are known to be strong at teaching advanced concepts, but little was known about how the choice of what to fade affects their difficulty in an advanced setting. We designed a framework for creating fading strategies, and used it to create and analyze three “context-aware” strategies in a between-groups study of intermediate CS students. While the strategies were essentially equivalent in terms of retention, difficulty, and students’ subjective preference, the neutrally context-aware, randomized baseline strategy required more student interactions. We therefore gently recommend against choosing a randomized strategy. Finally, since the study context required students to attempt a problem repeatedly until they solved it, these findings are most useful in formative, skill-building contexts.

Acknowledgments

This work was supported by grant #00005919 from the California Education Learning Lab (calearninglab.org), an initiative of the California Governor’s Office of Planning and Research.

References

- [1] Joe Betts, William Muntean, Doyoung Kim, and Shu-chuan Kao. 2022. Evaluating Different Scoring Methods for Multiple Response Items Providing Partial Credit. *Educational and Psychological Measurement* 82, 1 (Feb. 2022), 151–176. doi:10.1177/0013164421994636 Publisher: SAGE Publications Inc.
- [2] Serena Caraco, Nelson Lojo, Michael Verdicchio, and Armando Fox. 2024. Generating Multi-Part Autogradable Faded Parsons Problems From Code-Writing Exercises. In *Proceedings of the 55th ACM Technical Symposium on Computer Science*.

- Education V. 1* (Portland, OR, USA) (SIGCSE 2024). Association for Computing Machinery, New York, NY, USA, 179–185. doi:10.1145/3626252.3630786
- [3] Nick Cheng and Brian Harrington. 2017. The Code Mangler: Evaluating Coding Ability Without Writing Any Code. In *Proceedings of the 2017 ACM SIGCSE Technical Symposium on Computer Science Education - SIGCSE '17*. ACM Press, New York, NY, USA, 123–128. doi:10.1145/3017680.3017704
 - [4] Barbara J. Ericson, James D. Foley, and Jochen Rick. 2018. Evaluating the Efficiency and Effectiveness of Adaptive Parsons Problems. In *Proceedings of the 2018 ACM Conference on International Computing Education Research*. ACM, New York, NY, USA, 60–68. doi:10.1145/3230977.3231000
 - [5] Barbara J. Ericson, Lauren E. Margulieux, and Jochen Rick. 2017. Solving parsons problems versus fixing and writing code. In *Proceedings of the 17th Koli Calling Conference on Computing Education Research - Koli Calling '17*. ACM Press, New York, NY, USA, 20–29. doi:10.1145/3141880.3141895
 - [6] Gudmund S. Frandsen and Michael I. Schwartzbach. 2006. A singular choice for multiple choice. *SIGCSE Bull.* 38, 4 (June 2006), 34–38. doi:10.1145/1189136.1189164
 - [7] Flynn Fromont, Hiruna Jayamanne, and Paul Denny. 2023. Exploring the Difficulty of Faded Parsons Problems for Programming Education. In *Proceedings of the 25th Australasian Computing Education Conference (ACE '23)*. Association for Computing Machinery, New York, NY, USA, 113–122. doi:10.1145/3576123.3576136
 - [8] Sandra G. Hart and Lowell E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research. In *Human Mental Workload*, Peter A. Hancock and Najmedin Meshkati (Eds.). Advances in Psychology, Vol. 52. North-Holland, 139–183. doi:10.1016/S0166-4115(08)62386-9 ISSN: 0166-4115.
 - [9] Winston Haynes. 2013. Benjamini–Hochberg Method. In *Encyclopedia of Systems Biology*. Springer, New York, NY, 78–78. doi:10.1007/978-1-4419-9863-7_1215
 - [10] Petri Ihanntola and Ville Karavirta. 2010. Open source widget for parson's puzzles. In *Proceedings of the fifteenth annual conference on Innovation and technology in computer science education - ITiCSE '10*. ACM Press, New York, NY, USA, 302. doi:10.1145/1822090.1822178
 - [11] Nelson Lojo and Armando Fox. 2022. Teaching Test-Writing As a Variably-Scaffolded Programming Pattern. In *Proceedings of the 27th ACM Conference on Innovation and Technology in Computer Science Education Vol. 1 (ITiCSE '22)*. Association for Computing Machinery, New York, NY, USA, 498–504. doi:10.1145/3502718.3524789
 - [12] Dale Parsons and Patricia Haden. 2006. Parson's Programming Puzzles: A Fun and Effective Learning Tool for First Programming Courses. In *Proceedings of the 8th Australasian Conference on Computing Education - Volume 52 (ACE '06)*. Australian Computer Society, Inc., AUS, 157–163. event-place: Hobart, Australia.
 - [13] Nathaniel Weinman, Armando Fox, and Marti Hearst. 2020. Exploring Challenging Variations of Parsons Problems. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education*. ACM, New York, NY, USA, 1349. doi:10.1145/3328778.3372639
 - [14] Matthew West, Geoffrey L. Herman, and Craig Zilles. 2015. PrairieLearn: 2015 122nd ASEE Annual Conference and Exposition. *ASEE Annual Conference and Exposition, Conference Proceedings* 122nd ASEE Annual Conference and Exposition: Making Value for Society, 122nd ASEE Annual Conference and Exposition: Making Value for... (2015). <http://www.scopus.com/inward/record.url?scp=84941994154&partnerID=8YFLogxK>
 - [15] Zihan Wu. 2023. Investigating the Effectiveness of Variations of Micro Parsons Problems. In *Proceedings of the 2023 ACM Conference on International Computing Education Research - Volume 2* (Chicago, IL, USA) (ICER '23). Association for Computing Machinery, New York, NY, USA, 120–122. doi:10.1145/3568812.3603447
 - [16] Rui Zhi, Min Chi, Tiffany Barnes, and Thomas W. Price. 2019. Evaluating the Effectiveness of Parsons Problems for Block-based Programming. In *Proceedings of the 2019 ACM Conference on International Computing Education Research* (Toronto ON, Canada) (ICER '19). Association for Computing Machinery, New York, NY, USA, 51–59. doi:10.1145/3291279.3339419